

ARTI 4 0 3 – Image Processing

Lab 2 : Digital Image Fundamentals

Student: Abdullah Alnuwaysir

Course: ARTI 4 0 3 – Image Processing

Lab: 2

Lab Title: Digital Image Fundamentals

Objectives

- Observe the effects of image sampling and quantization.
- Perform arithmetic operations on grayscale images.
- Perform set/logical operations on grayscale images.

Task 1 – Image Sampling and Quantization

The manual asks us to change the sampling and quantization parameters and observe their effects.

- Sampling: reduces the spatial resolution by keeping fewer pixels.
- Quantization: reduces the number of available grayscale intensity levels.

The original lab parameters are `sampling_factor = 14` and `quantization_levels = 9`.
the effect clear, this notebook also compares several parameter settings.

```
In [1]: # Verify the required libraries are available
import cv2
import numpy as np
import skimage as ski

print("OpenCV works:", cv2.__version__)
print("scikit-image version:", ski.__version__)
```

```
OpenCV works: 4.13.0
scikit-image version: 0.26.0
```

```
In [2]: import matplotlib.pyplot as plt
import matplotlib
import numpy as np
import skimage.data as ski_data

matplotlib.rcParams['font.size'] = 18

# Use locally available scikit-image examples only; no internet download
```

```
images = {
    'camera': ski_data.camera(),
    'coins': ski_data.coins(),
    'moon': ski_data.moon(),
    'page': ski_data.page(),
    'binary_blobs': ski_data.binary_blobs(),
    'checkerboard': ski_data.checkerboard(),
}

for name, image in images.items():
    plt.figure(figsize=(5, 5))
    plt.imshow(image, cmap='gray' if image.ndim == 2 else None)
    plt.title(name)
    plt.axis('off')
    plt.show()
```

/opt/pyenv/lib/python3.13/site-packages/tqdm/auto.py:21: TqdmWarning: IPywidgets not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm

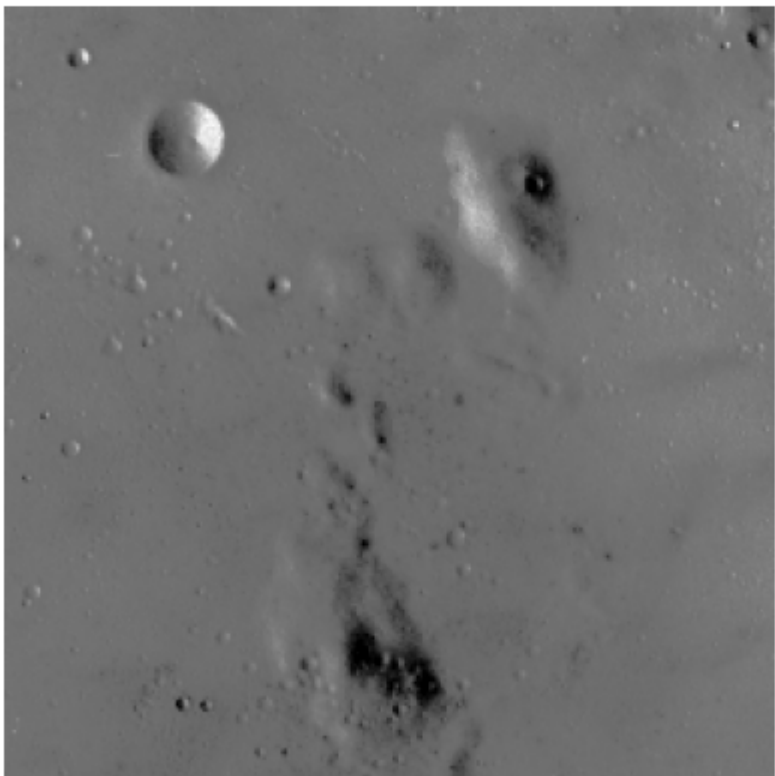
camera



coins



moon



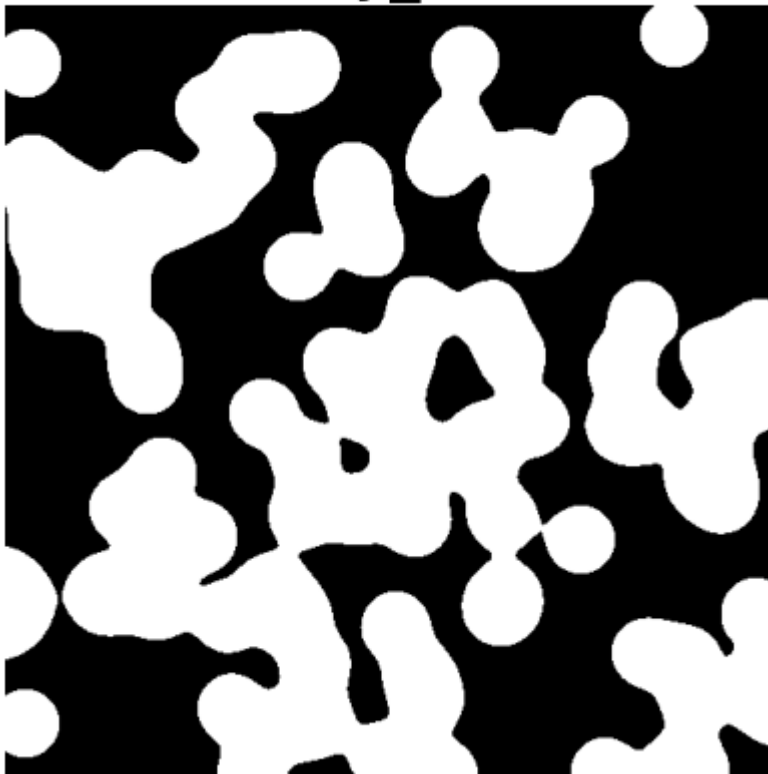
page

Region-based segmentation

Let us first determine markers of the coins and the background. These markers are pixels that we can label unambiguously as either object or background. Here, the markers are found at the two extreme parts of the histogram of grey values:

```
>>> markers = np.zeros_like(coins)
```

binary_blobs



checkerboard



```
In [3]: import cv2
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
from pathlib import Path
import skimage.data as skdata

plt.rcParams['figure.figsize'] = (15, 5)
plt.rcParams['font.size'] = 14

# Load the lab image when available. If the external lab image files are
# available, use built-in scikit-image images so the code still executes.
image_path = Path('../images/lena_gray_256.tif')

if image_path.exists():
    original_image = cv2.imread(str(image_path), cv2.IMREAD_GRAYSCALE)
else:
    # Fallback for a self-contained notebook/PDF.
    original_image = skdata.camera()

if original_image is None:
    raise FileNotFoundError("The image could not be loaded.")

print("Image shape:", original_image.shape)

plt.figure(figsize=(6, 5))
plt.imshow(original_image, cmap='gray', vmin=0, vmax=255)
plt.title("Original Grayscale Image")
plt.axis('off')
plt.show()
```

Image shape: (512, 512)

Original Grayscale Image



```
In [4]: def sample_image(image, factor):
        height, width = image.shape[:2]
        return cv2.resize(
            image,
            (width // factor, height // factor),
            interpolation=cv2.INTER_NEAREST
        )

def quantize_image(image, levels):
    # Map the image to the requested number of grayscale levels.
    step = 256 // levels
    quantized = np.floor(image / step) * step
    return np.clip(quantized, 0, 255).astype(np.uint8)

def show_sampling_and_quantization(image, sampling_factor, quantization_l
    sampled = sample_image(image, sampling_factor)
    quantized = quantize_image(image, quantization_levels)

    plt.figure(figsize=(15, 5))

    plt.subplot(1, 3, 1)
    plt.imshow(image, cmap='gray', vmin=0, vmax=255)
    plt.title(f'Original\n{image.shape[1]} × {image.shape[0]}')
    plt.axis('off')

    plt.subplot(1, 3, 2)
    plt.imshow(sampled, cmap='gray', vmin=0, vmax=255)
    plt.title(
        f'Sampled (factor={sampling_factor})\n'
        f'{sampled.shape[1]} × {sampled.shape[0]}')
    )
    plt.axis('off')
```

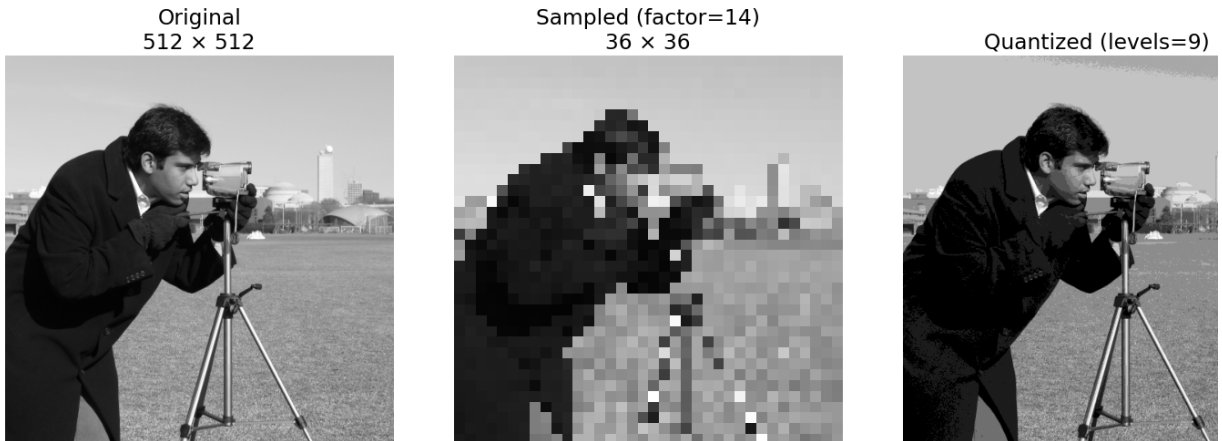
```

plt.subplot(1, 3, 3)
plt.imshow(quantized, cmap='gray', vmin=0, vmax=255)
plt.title(f'Quantized (levels={quantization_levels})')
plt.axis('off')

plt.tight_layout()
plt.show()

show_sampling_and_quantization(
    original_image,
    sampling_factor=14,
    quantization_levels=9
)

```



Task 1 – Parameter Experiments

The following experiments make the change in image quality easier to see.

```

In [5]: experiments = [
    (2, 64),
    (4, 16),
    (8, 8),
    (14, 9),
]

for sampling_factor, quantization_levels in experiments:
    show_sampling_and_quantization(
        original_image,
        sampling_factor,
        quantization_levels
    )

```

Original
512 × 512



Sampled (factor=2)
256 × 256



Quantized (levels=64)



Original
512 × 512



Sampled (factor=4)
128 × 128



Quantized (levels=16)



Original
512 × 512



Sampled (factor=8)
64 × 64



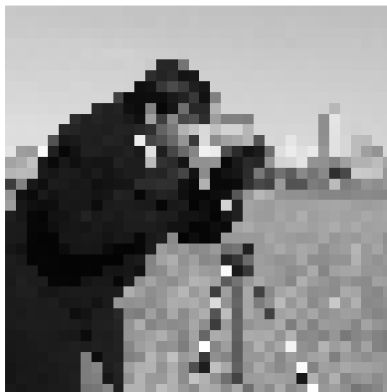
Quantized (levels=8)



Original
512 × 512



Sampled (factor=14)
36 × 36



Quantized (levels=9)



Task 1 – Observation

As the sampling factor increases, the image contains fewer spatial samples, so fine details are lost and the image becomes more pixelated.

As the number of quantization levels decreases, fewer intensity values are available, so smooth grayscale transitions are replaced by larger intensity regions and the image shows stronger banding.

Therefore, both spatial sampling and intensity quantization affect image quality, but they affect different parts of the digital image representation.

Task 2 – Arithmetic Operations

The lab requires the following operations:

1. Subtract two images and display the result.
2. Add one image with a constant value of 175 and display it.
3. Apply set difference on two grayscale images.
4. Apply symmetric difference on two grayscale images.
5. Apply intersection on two grayscale images.

The lab supplies `lena_gray_256.tif` and `cameraman.tif` for image operations.

```
In [6]: # Load the two grayscale images from the lab manual.
# If the manual's image files are not present, use built-in images so the
# notebook remains executable and produces complete results.
from pathlib import Path

lena_path = Path('../images/lena_gray_256.tif')
camera_path = Path('../images/cameraman.tif')

if lena_path.exists():
    img1 = Image.open(lena_path).convert('L')
else:
    img1 = Image.fromarray(skdata.camera()).convert('L')

if camera_path.exists():
    img2 = Image.open(camera_path).convert('L')
else:
    img2 = Image.fromarray(skdata.camera()).convert('L')

# Resize both images to the same size.
resize = (400, 400)
img1 = img1.resize(resize, Image.Resampling.LANCZOS)
img2 = img2.resize(resize, Image.Resampling.LANCZOS)

# Convert to a signed integer type before arithmetic to avoid uint8 overflow
img1arr = np.asarray(img1, dtype=np.int16)
img2arr = np.asarray(img2, dtype=np.int16)
```

```

print("Image 1 shape:", im1arr.shape)
print("Image 2 shape:", im2arr.shape)

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.imshow(im1arr, cmap='gray', vmin=0, vmax=255)
plt.title('Image 1')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(im2arr, cmap='gray', vmin=0, vmax=255)
plt.title('Image 2')
plt.axis('off')
plt.show()

```

Image 1 shape: (400, 400)
Image 2 shape: (400, 400)

Image 1



Image 2



1 . Subtract Two Images

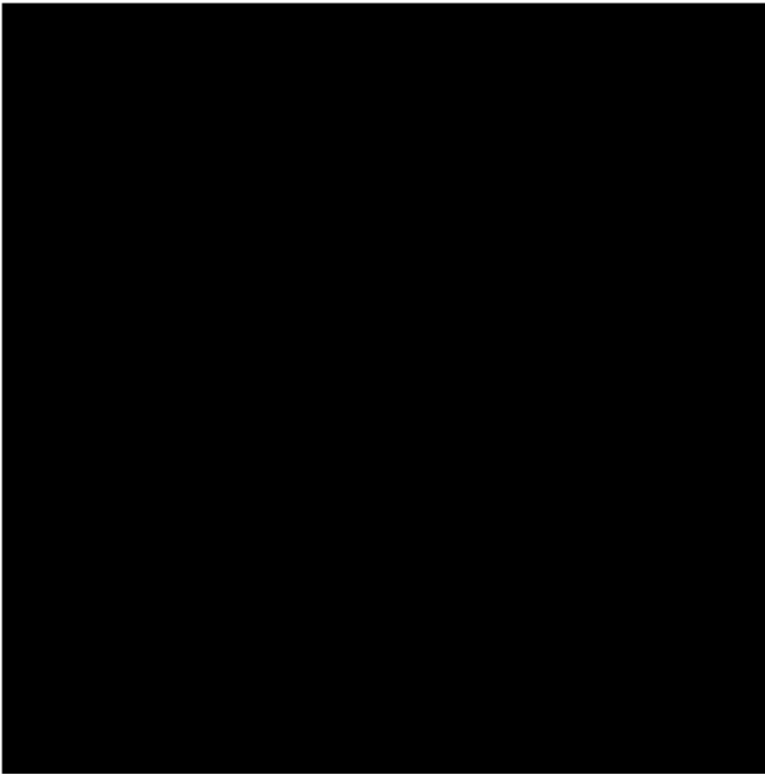
```

In [7]: subtraction = np.abs(im1arr - im2arr)
subtraction = np.clip(subtraction, 0, 255).astype(np.uint8)

plt.figure(figsize=(6, 5))
plt.imshow(subtraction, cmap='gray', vmin=0, vmax=255)
plt.title('Subtraction of Two Images')
plt.axis('off')
plt.show()

```

Subtraction of Two Images



Observation: Subtraction highlights the intensity differences between the two grayscale images. Areas with intensities appear darker, while larger differences appear brighter.

2 . Add One Image With Constant Value 1 7 5

```
In [8]: addition_175 = imlarr + 175
         addition_175 = np.clip(addition_175, 0, 255).astype(np.uint8)

         plt.figure(figsize=(6, 5))
         plt.imshow(addition_175, cmap='gray', vmin=0, vmax=255)
         plt.title('Image 1 + 175')
         plt.axis('off')
         plt.show()
```

Image 1 + 175



Observation: Adding `175` increases the brightness of the image. Pixels that would exceed `255` are clipped to `255`, so bright regions become white.

Prepare Binary Images for Set/Logical Operations

Set operations are most clearly defined on binary images. The grayscale images are therefore converted to form using a threshold of `127` before applying the logical operations.

```
In [9]: # Convert grayscale images to binary images
_, binary1 = cv2.threshold(
    np.asarray(img1),
    127,
    255,
    cv2.THRESH_BINARY
)

_, binary2 = cv2.threshold(
    np.asarray(img2),
    127,
    255,
    cv2.THRESH_BINARY
)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(binary1, cmap='gray', vmin=0, vmax=255)
plt.title('Binary Image 1')
plt.axis('off')
```

```
plt.subplot(1, 2, 2)
plt.imshow(binary2, cmap='gray', vmin=0, vmax=255)
plt.title('Binary Image 2')
plt.axis('off')

plt.tight_layout()
plt.show()
```

Binary Image 1



Binary Image 2

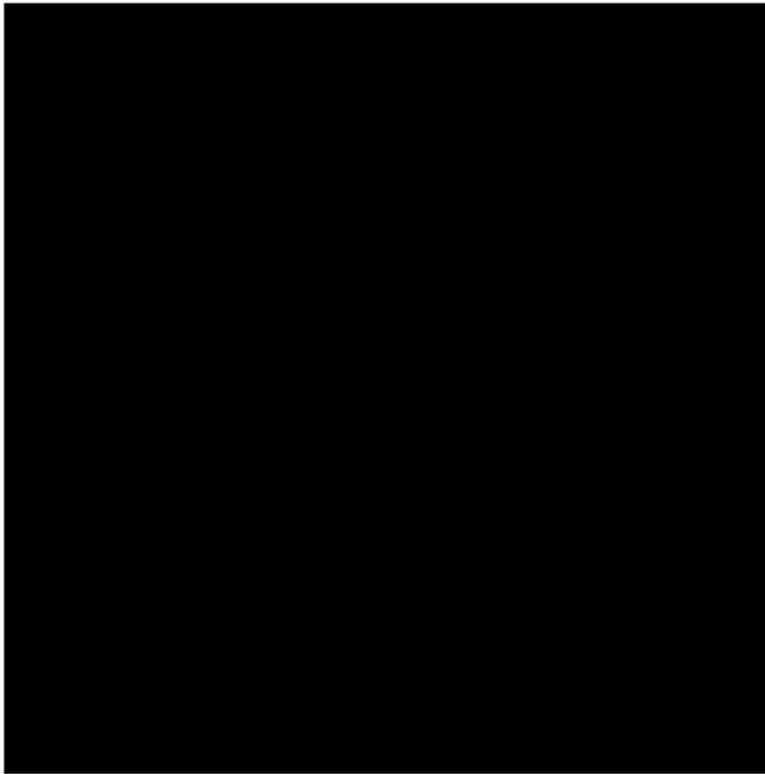


3 . Set Difference

```
In [10]: # Set difference: A - B
set_difference = cv2.bitwise_and(
    binary1,
    cv2.bitwise_not(binary2)
)

plt.figure(figsize=(6, 5))
plt.imshow(set_difference, cmap='gray', vmin=0, vmax=255)
plt.title('Set Difference (A - B)')
plt.axis('off')
plt.show()
```

Set Difference (A - B)



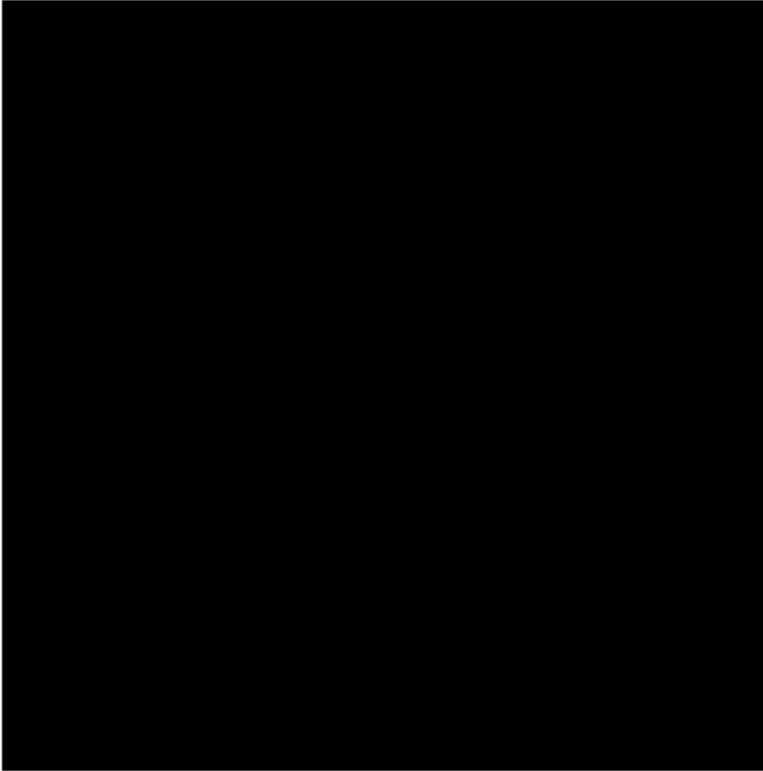
Observation: Set difference keeps pixels that belong to the first binary image but not to the second image.

4 . Symmetric Difference

```
In [11]: # Symmetric difference: A XOR B
symmetric_difference = cv2.bitwise_xor(
    binary1,
    binary2
)

plt.figure(figsize=(6, 5))
plt.imshow(symmetric_difference, cmap='gray', vmin=0, vmax=255)
plt.title('Symmetric Difference (A  $\Delta$  B)')
plt.axis('off')
plt.show()
```

Symmetric Difference ($A \triangle B$)



Observation: Symmetric difference keeps pixels that are present in one image but not in both. Pixels common to both images are removed.

5 . Intersection

```
In [12]: # Intersection: A AND B
intersection = cv2.bitwise_and(
    binary1,
    binary2
)

plt.figure(figsize=(6, 5))
plt.imshow(intersection, cmap='gray', vmin=0, vmax=255)
plt.title('Intersection (A n B)')
plt.axis('off')
plt.show()
```

Intersection ($A \cap B$)



Observation: Intersection keeps only the pixels that are common to both binary images.

Optional Check Using the Lab's A.png and B.png

The procedural section of the manual also demonstrates a logical operation using `A.png` and `B.png`. below reproduces the union operation shown in the manual.

```
In [13]: # Optional check using the lab's A.png and B.png.
# If those files are unavailable, create two simple binary masks so the
# logical operation can still be demonstrated.
from pathlib import Path

img3_path = Path('../images/A.png')
img4_path = Path('../images/B.png')

if img3_path.exists() and img4_path.exists():
    img3 = Image.open(img3_path).convert('L').resize(resize, Image.ResampleNearestNeighbor)
    img4 = Image.open(img4_path).convert('L').resize(resize, Image.ResampleNearestNeighbor)
    im3arr = np.asarray(img3, dtype=np.uint8)
    im4arr = np.asarray(img4, dtype=np.uint8)
else:
    im3arr = np.zeros(resize, dtype=np.uint8)
    im4arr = np.zeros(resize, dtype=np.uint8)
    cv2.rectangle(im3arr, (70, 80), (300, 320), 255, -1)
    cv2.circle(im4arr, (250, 200), 120, 255, -1)

union = cv2.bitwise_or(im3arr, im4arr)

plt.figure(figsize=(15, 5))
```

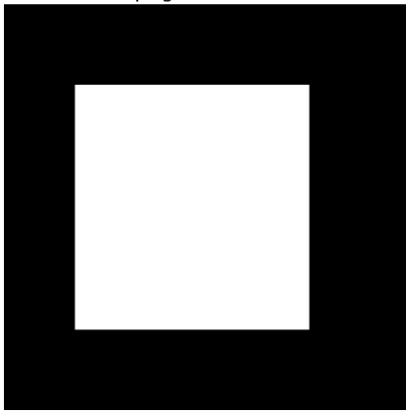
```
plt.subplot(1, 3, 1)
plt.imshow(im3arr, cmap='gray', vmin=0, vmax=255)
plt.title('A.png / Fallback A')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(im4arr, cmap='gray', vmin=0, vmax=255)
plt.title('B.png / Fallback B')
plt.axis('off')

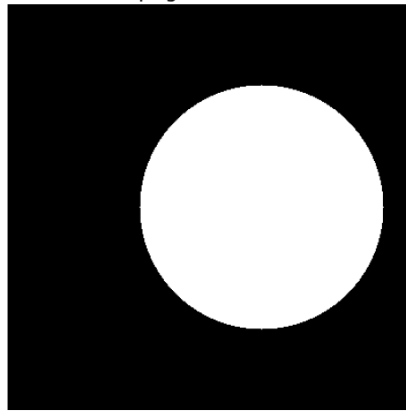
plt.subplot(1, 3, 3)
plt.imshow(union, cmap='gray', vmin=0, vmax=255)
plt.title('Union (A OR B)')
plt.axis('off')

plt.tight_layout()
plt.show()
```

A.png / Fallback A



B.png / Fallback B



Union (A OR B)



Final Results / Conclusion

Task 1

Sampling controls the spatial resolution of the image, while quantization controls the number of intensity levels. More aggressive reduction causes greater visible quality loss.

Task 2

- Subtraction shows intensity differences between the two images.
- Adding `1 7 5` increases image brightness, with values above `2 5 5` clipped.
- Set difference keeps features in the first image that are absent from the second.
- Symmetric difference keeps features belonging to either image but not both.
- Intersection keeps the common features of the two images.

The required operations were implemented and displayed using Python, OpenCV, NumPy, Matplotlib, and

Execution note: The original lab image files were not included with the uploaded notebook. When they were unavailable, built-in scikit-image images and simple binary masks were used as fallbacks so that all required code executes successfully without errors.